

Data Structures - Sample Midterm Exam

1. Mark by T(=true) or F(=False) each of the following statements. You don't need to prove it.

✓ (1) $(6n + 4)(1 + \log n) = \Theta(n \log n)$

✗ (2) $n^n = O(2^n)$

✓ (3) $2^n = O(n!)$

✓ (4) $\sum_{i=1}^n i \log i = \Theta(n^2 \log n)$

✗ (5) If $f(n) = \Theta(g(n))$, then $2^{f(n)} = \Theta(2^{g(n)})$.

✗ (6) If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then the difference $d(n) - e(n)$ is $O(f(n) - g(n))$.

(4) $0 + 2 \cdot \log 2 + \dots + n \log n$

$a_0 = 0 \quad a_n = n \log n \quad \text{length} = n$

$\text{avg.} = \frac{(0 + n \log n)}{2} \cdot n = \frac{1}{2} n^2 \log n$

(5) Let $f(n) = 100n \quad g(n) = n$

$C_1 2^n \leq 2^{100n} \leq C_2 \cdot 2^n$

$C_1 = 1$

$C_2 = \infty$

(6) Let $f(n) = 2n \quad g(n) = n$

$d(n) = (af(n)) = n \quad e(n) = O(g(n)) = n$

$d(n) - e(n) \neq O(f(n) - g(n))$

$n - n \neq O(2n - n)$

$0 \neq O(n)$

2. Please find the most appropriate order of growth of the followings with big-O notation:

(1) $n^3 2^n + 6n^2 3^n \quad A: O(n^2 3^n)$

(2) $\sum_{i=1}^n i^3 \quad A: O(n^4)$

(3) $\sum_{i=2}^{n-1} \log i^2 \quad A: O(n \log n)$

(4) $\sum_{i=1}^n (i+1) \times 2^{i-1} \quad A: O(n \cdot 2^n)$

(2) $\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4} = O(n^4)$

(3) $\sum_{i=2}^{n-1} \log i^2 = 2 \sum_{i=2}^{n-1} \log i = 2 \cdot (\log 2 + \log 3 + \dots + \log (n-1))$

$\leq 2 \cdot (\log (n-1) + \log (n-1) + \dots + \log (n-1))$

$= 2 \cdot \log (n-1) \cdot (n-2) = O(n \log n)$

(4) $\sum_{i=1}^n (i+1) \times 2^{i-1} = \underbrace{\frac{1}{2} \sum_{i=1}^n i 2^i} + \frac{1}{2} \sum_{i=1}^n 2^i$

$\hookrightarrow = \frac{n \cdot 2^{n+2} - (n+1) \cdot 2^{n+1} + 2}{(2-1)^2}$

$= \underbrace{4n \cdot 2^n - 2n \cdot 2^n - 2 \cdot 2^n + 2}_{= O(n 2^n)}$

3. Consider two functions $f(n) = 3n^2$ and $g(n) = 33n + 36$. Determine n_0 such that f is larger than g for $n \geq n_0$.

$$f(n) = g(n)$$

$$3n^2 - 33n - 36 = 0$$

$$n = \underline{\underline{12}} \text{ or } -1$$

$$A: n_0 = 12$$

4. There are three clubs *Astronomy*, *Baseball*, and *Chorus* and each student can select at most two clubs to participate in. The school thus needs to verify this for three clubs. Suppose the school has the member lists of these three clubs, say A , B , and C and each list is of size n .

- (a) Suppose the school uses the following code `disjoint1()` to perform the verification please analyze the time complexity of the given code.

```
def disjoint1(A, B, C):  
    for a in A:  $n$   
        for b in B:  $n \cdot n$   
            for c in C:  $n \cdot n \cdot n$   
                if a == b == c:  $n \cdot n \cdot n \cdot 3$   
                return False  
    return True
```

$$A: O(n^3)$$

- (b) However, the school also uses the following code `disjoint2()` to process. Please check the complexity of the new code and explain the difference between these two pieces of code.

```
def disjoint2(A, B, C):  
    for a in A:  $n$   
        for b in B:  $n \cdot n$   
            if a == b:  
                for c in C:  $n \cdot n \cdot n$   
                    if a == c:  
                        return False  
    return True
```

$$A: O(n^3), \text{ 有剪支}$$

5. In the following problems, select a big-O notation among $O(n)$, $O(\log n)$, $O(n \log n)$, $O(\log \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, or $O(n!)$ for the number of times the statement $x = x + 1$ is executed.

(a)

```
def Mid_Exam_sample(n):
    x=0
    for i in range(n):
        for j in range(n):
            x = x + 1
```

$A: O(n^2)$

6. Please consider the following pseudo-code and answer the questions, where A is an $n \times m$ matrix.

Input array $A[n][m]$

Create array $B[m][n]$

```
for i = 1 to n
    for j = 1 to m
        B[j][i] = A[i][j]
```

- Please state what this pseudo-code does.
- Please give the number of operations (steps) for each statement. Then, sum up these numbers to derive the total number of operations.
- What is the asymptotic time of this program?

(a) 將 array A 以左上一右下對角線翻轉的結果存入 array B

(b) $1+n + n \cdot (1+m) + 2n \cdot (1+m)$

$= 1+n + 3n(1+m)$

(c) $O(m \cdot n)$

7. Recall the *inversion number* of a permutation in the homework. Please prove by *induction* that the maximum number of inversions for a permutation P with size n is $\frac{n(n-1)}{2}$.

$$\text{Let } P(n) = \frac{n(n-1)}{2}$$

(1) Base Case:

$$P(1) = \frac{1 \cdot (1-1)}{2} = 0$$

$$(2) P(i) = \frac{i \cdot (i-1)}{2}$$

$$\begin{aligned} (3) P(i+1) &= \frac{(i+1) \cdot i}{2} \\ &= \frac{(i-1) \cdot i}{2} + i \\ &= P(i) + i \end{aligned}$$

$$P_n = \frac{n(n-1)}{2}$$